

CHAPTER-4-ADVERSARIAL SEARCH

Wattanapong Suttapak

2 OUTLINE

- Adversarial Search
- Games' rules
- Brute force algorithm
- Minimax algorithm

3 ADVERSARIAL SEARCH

- เป็นเทคนิคที่ใช้ในการค้นหาคำตอบของเกมสักระหว่าง 2 ผู้เล่น
- เรียกอีกชื่อว่า Game search technique
- เทคนิคนี้จะสามารถนำไปใช้แก้ปัญหาได้ ผู้เล่นต้องสามารถคาดการณ์ หรือพยากรณ์ความน่าจะเป็นของการเดินทั้งหมดได้
 - เช่น หมากรอส หมากรุก Go รวมถึงเกมสัประเภทวางแผน (strategy)
 - ผู้เล่นที่หนึ่ง สามารถคาดการณ์การเดินของคู่ต่อสู้ หลังจากการเดินหมากรุกในแต่ละครั้งได้
- การค้นหาโอกาสชนะ จะมีความยาก และซับซ้อนขึ้น เนื่องจากคู่ต่อสู้ก็ต้องการค้นหาโอกาสชนะด้วย

4 ADVERSARIAL SEARCH

- ในบทนี้เรากำลังพูดถึงการสร้างเกมส์โดยใช้เทคนิคด้าน AI ด้วยวิธีการ **combinatorial search**
- **combinatorial search = brute-force approach ~ backtracking**
- เป็นวิธีการสำรวจทุกความน่าจะเป็น ทุกความเป็นไปได้ ในการค้นหา
Example algorithms; depth-first search, breadth-first search
- Need **recursive function**
- Strengthen: Global optimum
- Weakness: Impossible to explore all states, consume resources and time

5 ADVERSARIAL SEARCH

- ตัวอย่างเกมส์ประเภท 2 ผู้เล่น (2-player games)
- เป้าหมาย:
 - ลดความน่าจะเป็นที่ทำให้โอกาสเข้าสู่กรณีที่แย่ที่สุด (worst-case scenario)
 - สมมติให้คู่แข่งเล่นได้ดีที่สุด (**optimally**)
- ตัวอย่างเกมส์
 - Tic-Tac-Toe, Hexapawn
- Algorithms
 - The Minimax algorithm, alpha-Beta pruning, algorithm negamax algorithm

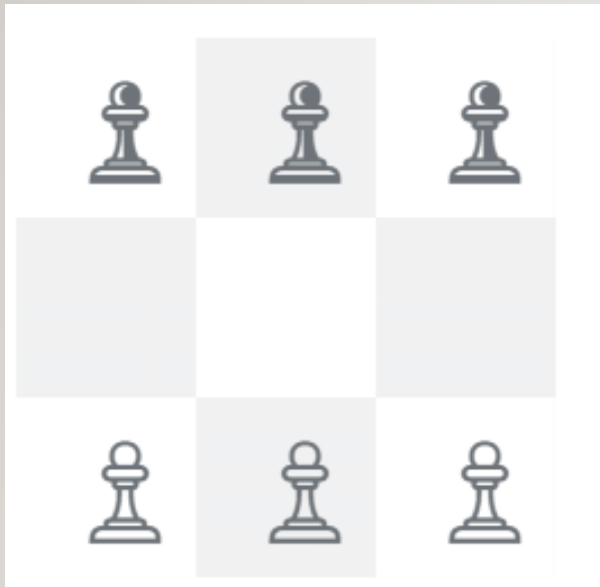
6 TIC-TAC-TOE

X	X	O
O	O	X
X		

Rule:

- ผู้เล่นสลับเติม 1 เครื่องหมาย ต่อครั้ง
- สามารถเติมเครื่องหมาย “X” หรือ “O” บนตาราง 3x3
- ผู้ชนะ:
 - สัญลักษณ์เดียวกันอยู่บนแถว หรือคอลัมน์ หรือแนวทแยง
- เสมอ
 - เมื่อเติมเครื่องหมายเต็ม 9 ช่อง แต่ไม่มีผู้ชนะ

7 HEXAPAWN



Rule:

- ผู้เล่นย้ายได้หนึ่งช่องต่อครั้ง สลับกันเดิน
- ทิศทางการเดินเป็นไปได้ 2 วิธี
 - เดินตรงหนึ่งช่อง
 - ดึงตัวเดิน (pawn) คู่ต่อสู้ออกจากเกมส์ได้ หากตัวเดินคู่ต่อสู้อยู่แนวทแยง
- **ผู้ชนะ:**
 - ถ้าตัวเดินดึงไปถึงฝั่งตรงข้ามได้หนึ่งตัว
 - ยึดทุกตัวคู่ต่อสู้ได้
 - ป้องกันคู่ต่อสู้ไม่ให้เดิน

สามารถปรับขนาดตารางได้: Grid size 3x3, 4x4, 8x8, nxn

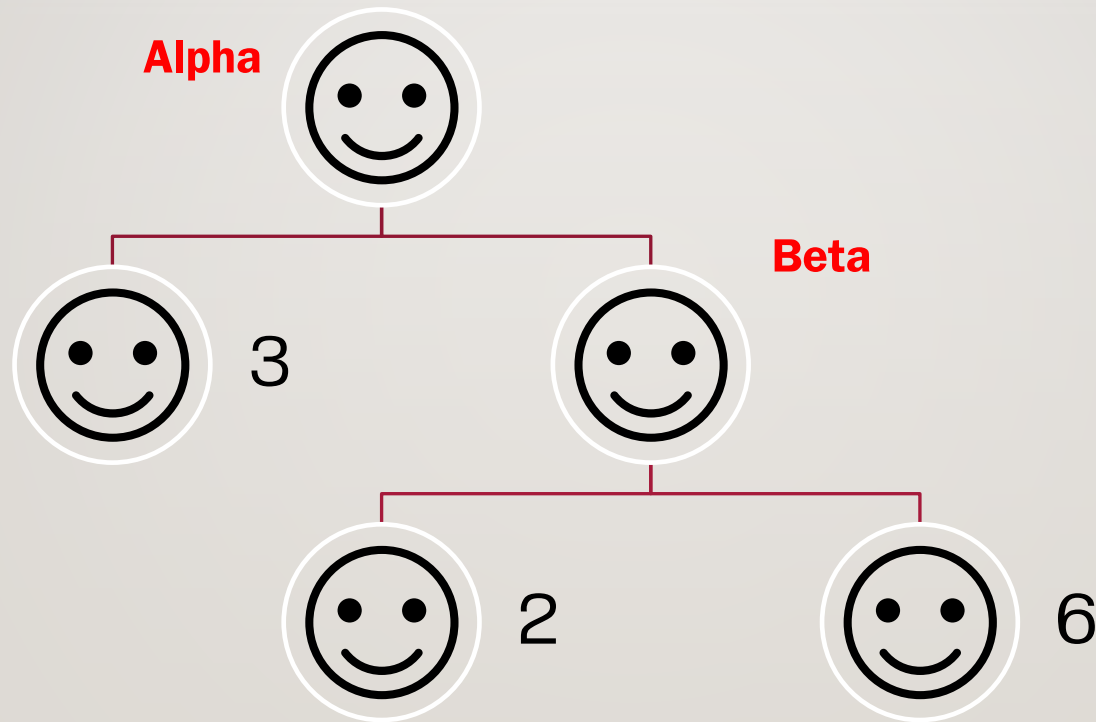
8 MINIMAX ALGORITHM

- When two players are playing against each other
- Each side needs to **predict** what the **opponent** is **going to do** in order to **win** the game
- **Minimax** tries to **minimize** the function that the **opponent** is trying to **maximize**
- (minimax พยายามหาเส้นทางที่ทำให้คู่ต่อสู้มีโอกาสชนะลดลง)

9 ALPHA-BETA PRUNING

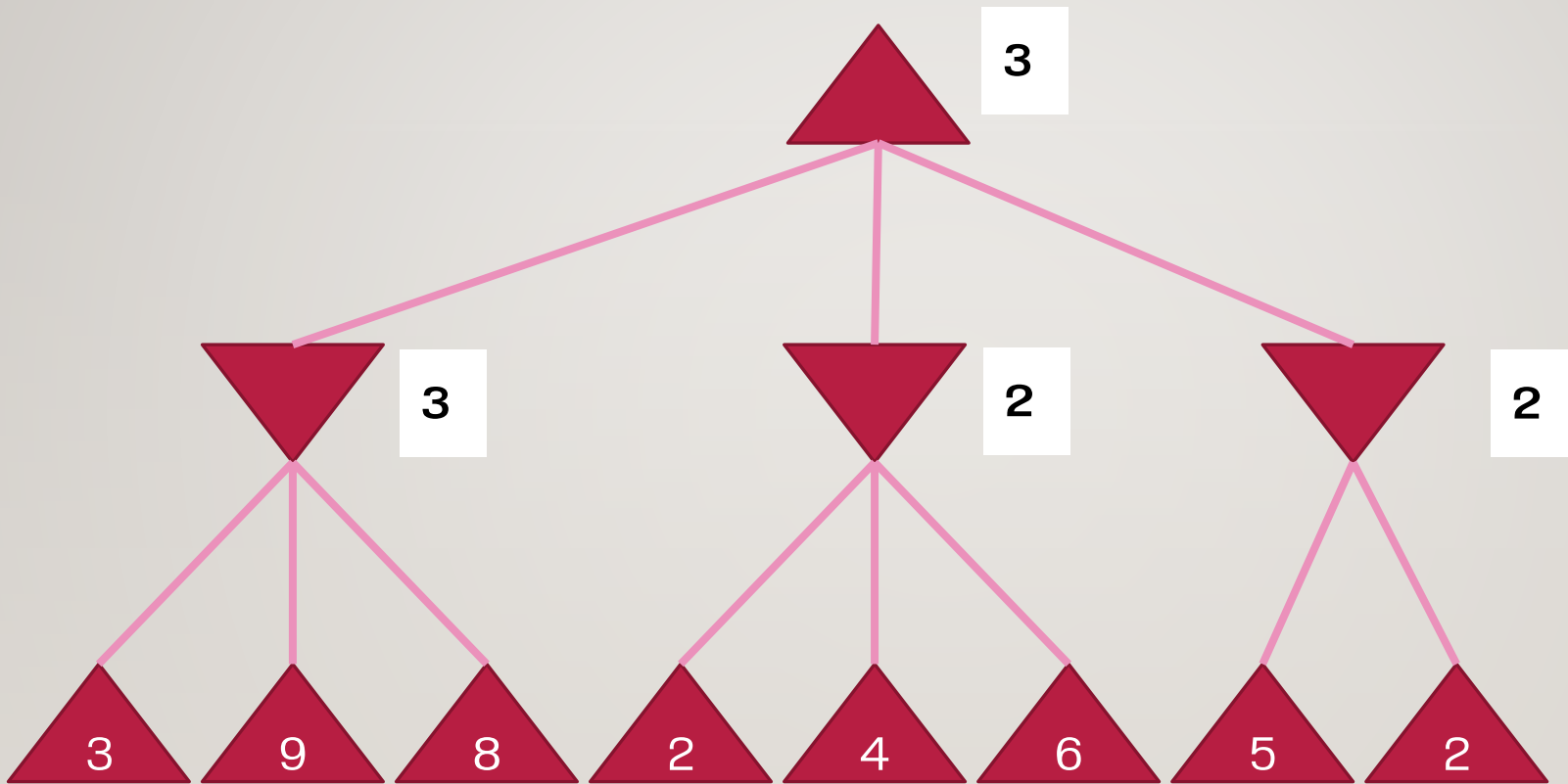
- สร้างขึ้นเพื่อพัฒนา Minimax Algorithm
- เป็นวิธีการค้นหาคำตอบโดยหลีกเลี่ยงเส้นทางที่ไม่สามารถหาคำตอบได้
- เราเรียกกระบวนการนี้ว่า **pruning**
- โดยหลักการ สามารถแบ่งเป็น 2 ส่วน
- Alpha จำลองเหตุการณ์คู่ต่อสู้ที่พยายามจะชนะในเกมส์ (Maximize winning chance)
- Beta จำลองเหตุการณ์ลดโอกาสชนะลง

10 ALPHA-BETA PRUNING

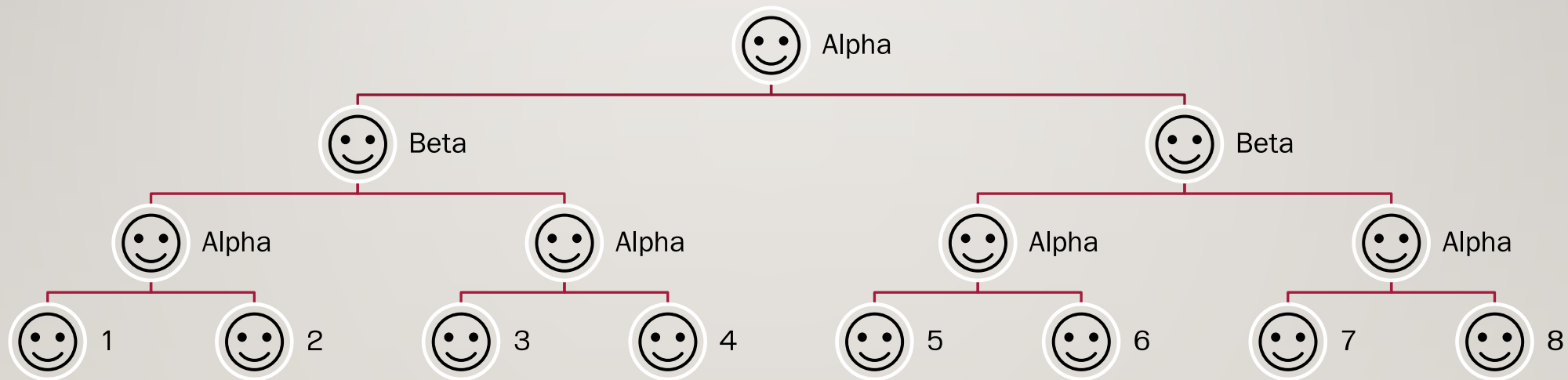


1. หา Max left branch = 3
 - 1) หา Min right branch
 - Left node เท่ากับ 2
 - 2) Min right = 2
2. เปรียบเทียบค่า $\max(3, 2) = 3$
3. Prune right branch
เนื่องจากไม่มีจำนวนเต็มใด ๆ
มากกว่า 3 แต่น้อยกว่า 2

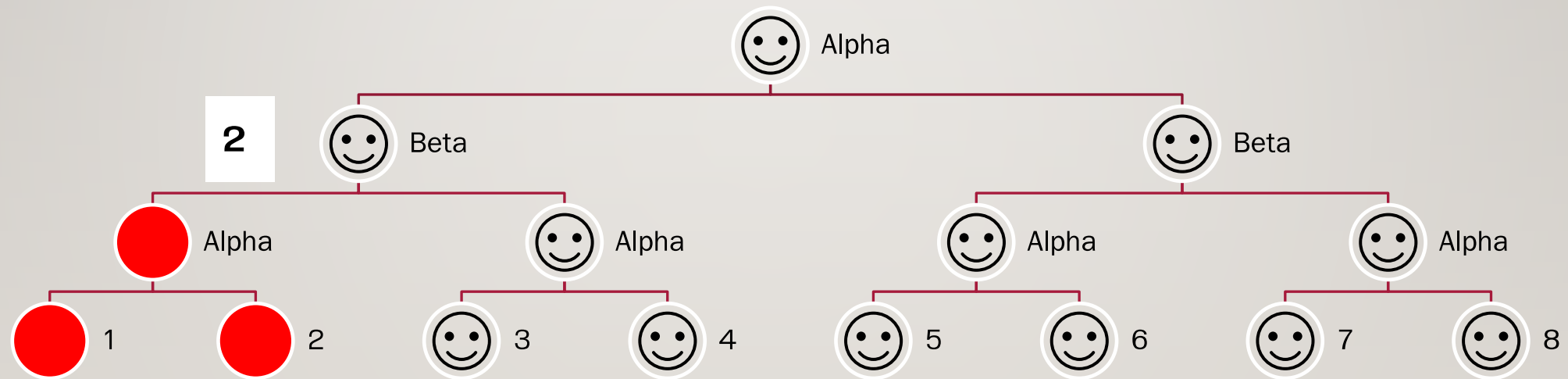
วิธีการเลือกเส้นทางของ ALPHA-BETA PRUNING



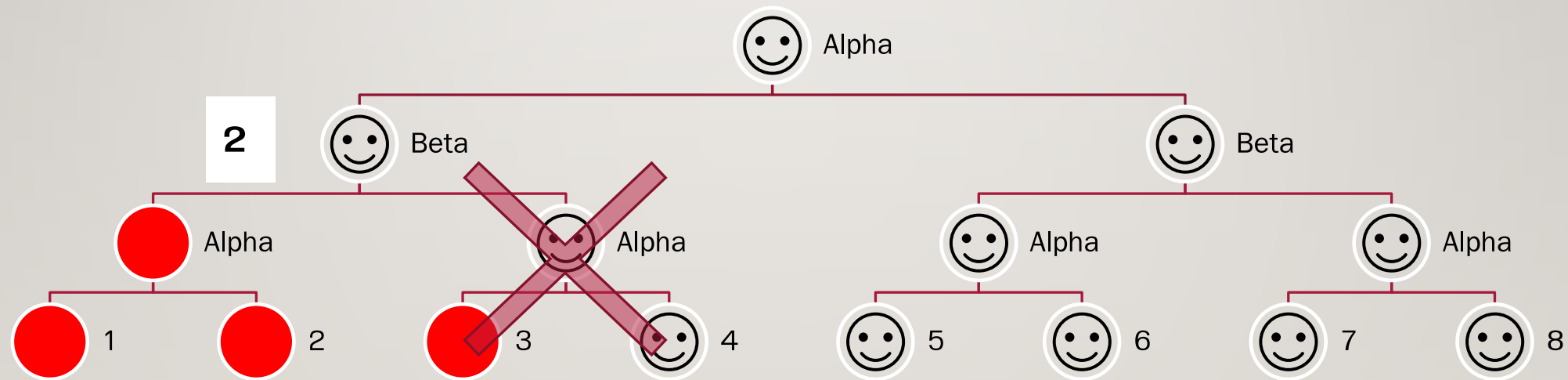
วิธีการเลือกเส้นทางของ ALPHA-BETA PRUNING



วิธีการเลือกเส้นทางของ ALPHA-BETA PRUNING

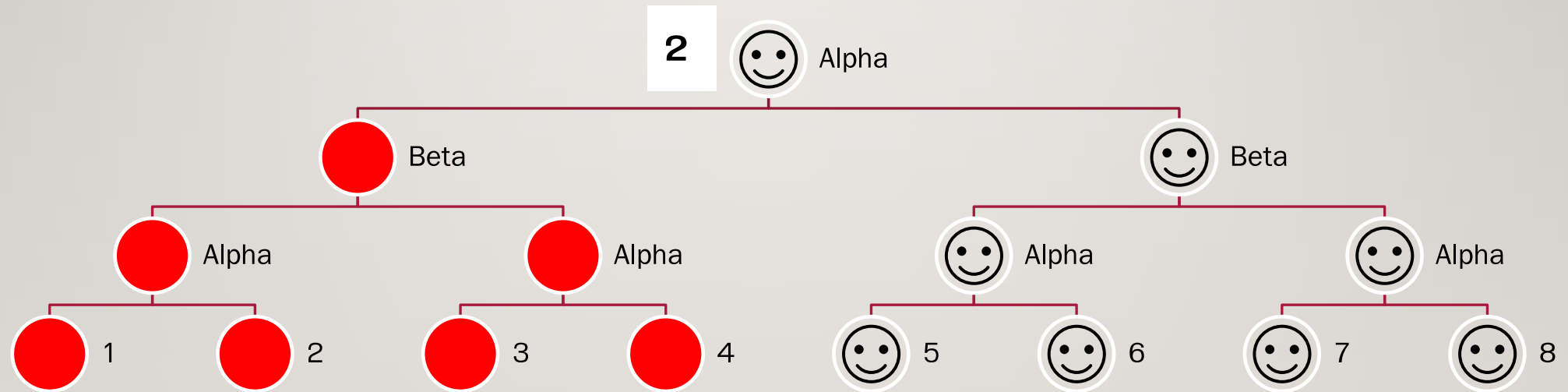


วิธีการเลือกเส้นทางของ ALPHA-BETA PRUNING

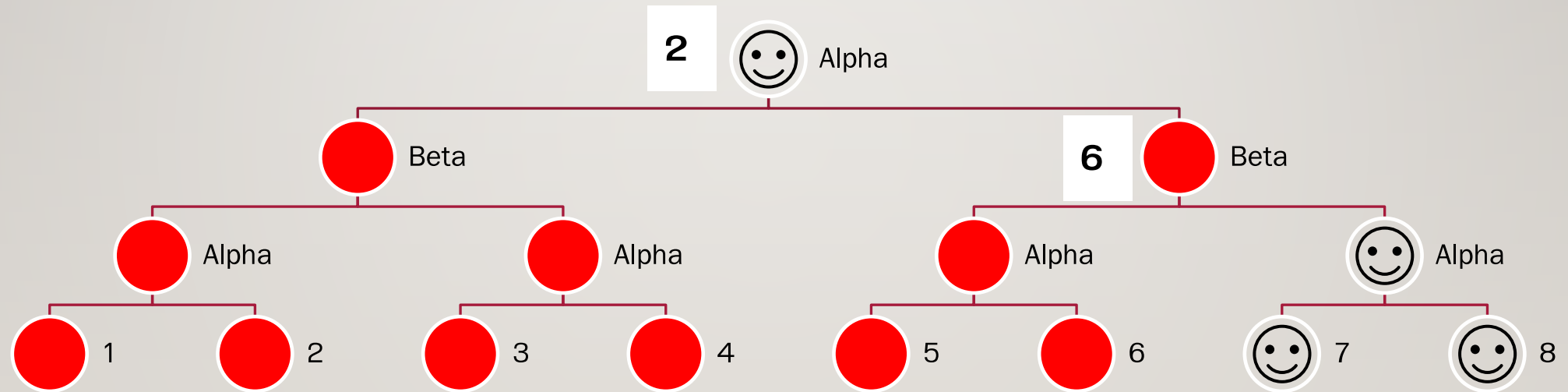


$$\max(3, n) < 2$$
$$n = \emptyset$$

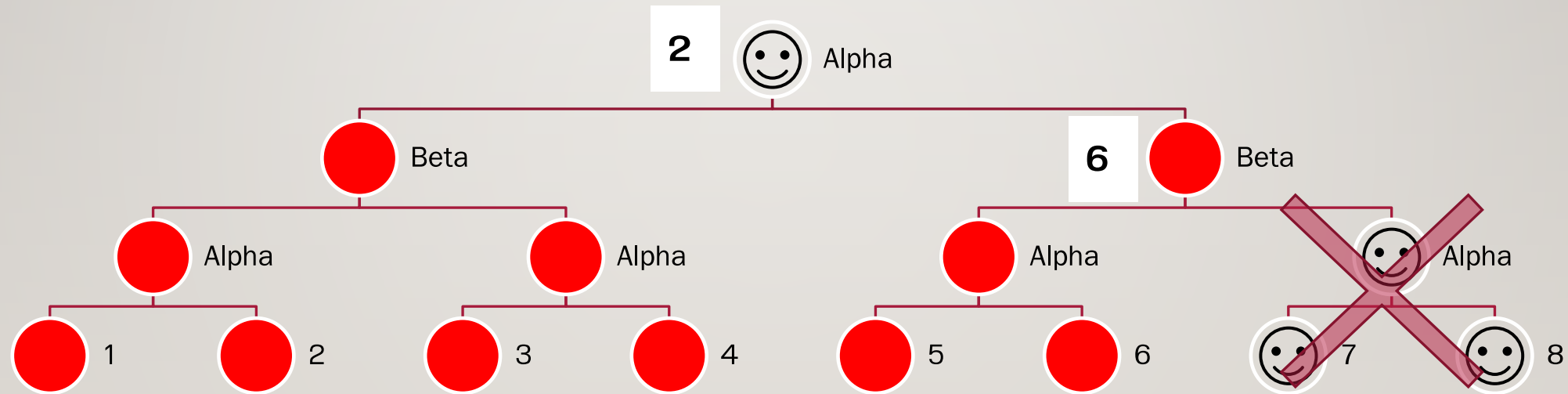
วิธีการเลือกเส้นทางของ ALPHA-BETA PRUNING



วิธีการเลือกเส้นทางของ ALPHA-BETA PRUNING



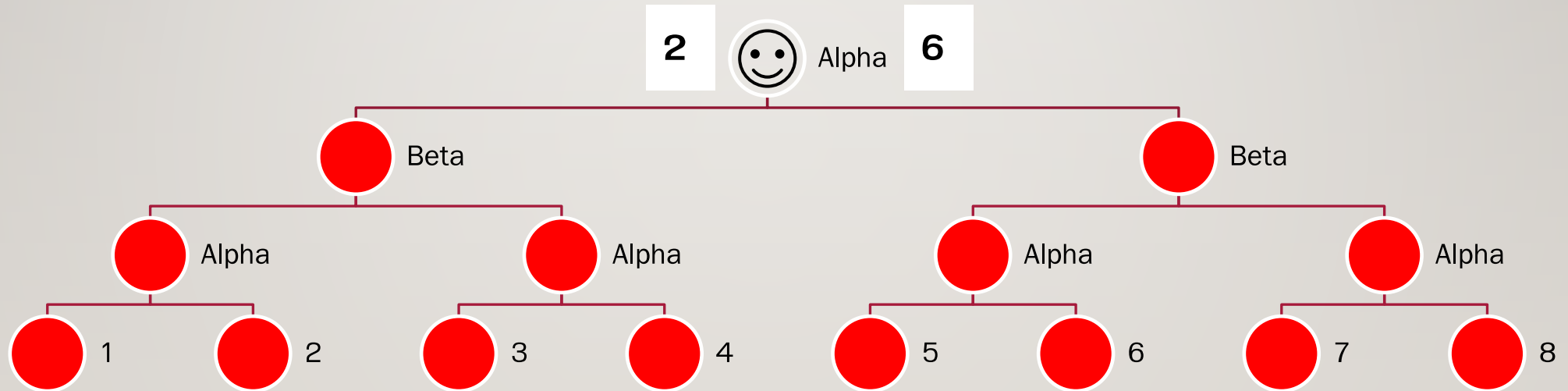
วิธีการเลือกเส้นทางของ ALPHA-BETA PRUNING



$$\max(7, n) < 6$$
$$n = \emptyset$$

วิธีการเลือกเส้นทางของ ALPHA-BETA PRUNING

$$\max(2, 6) = 6$$

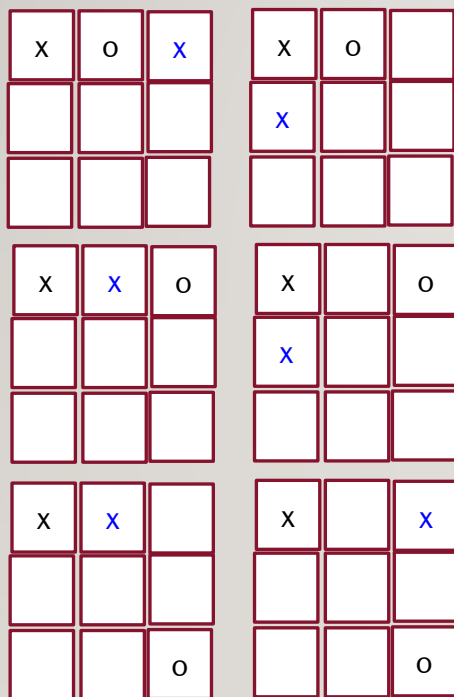


19 SOLVING TIC-TAC-TOE USING MINIMAX

1. Generate empty board 3x3 grids
2. Loop 9 times:
 1. Place move position : 'X'
 2. Find best solution using minimax
3. Check winner

20

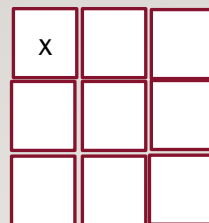
maximize



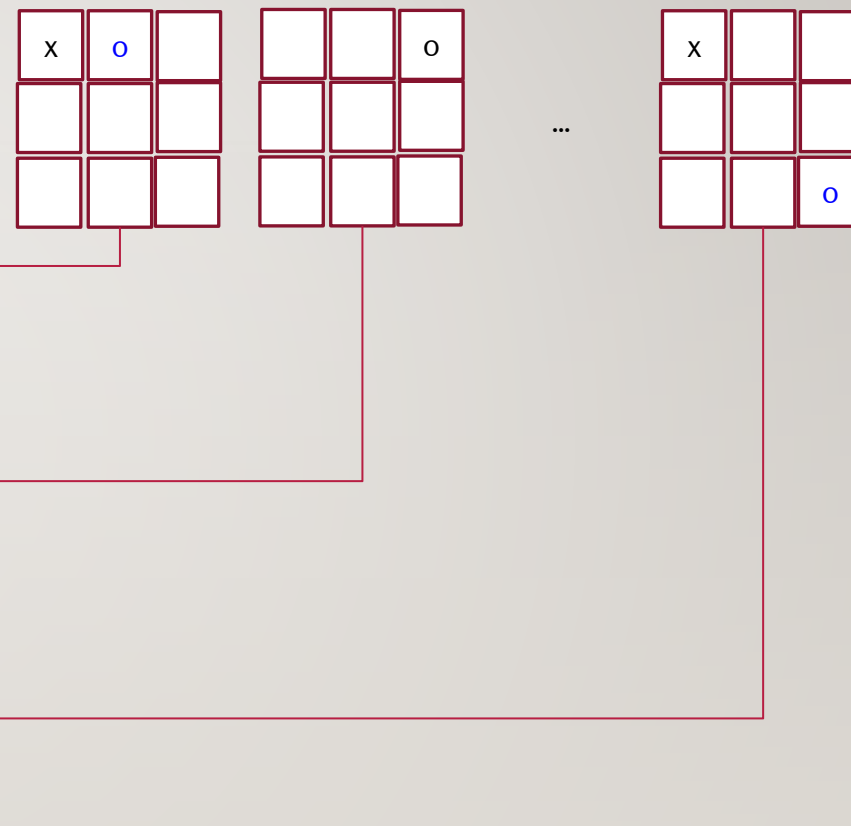
...

...

...



minimize



...

...

...

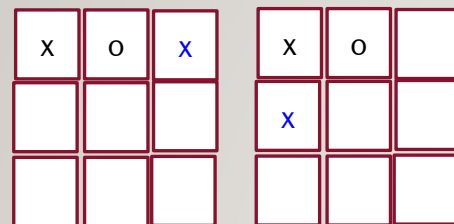
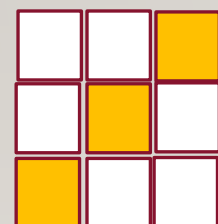
Minimax Function

Minimax Function

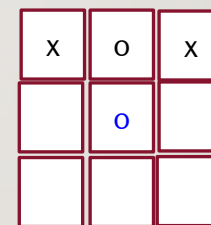
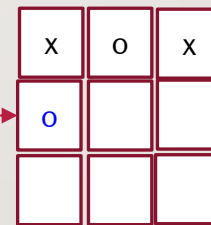
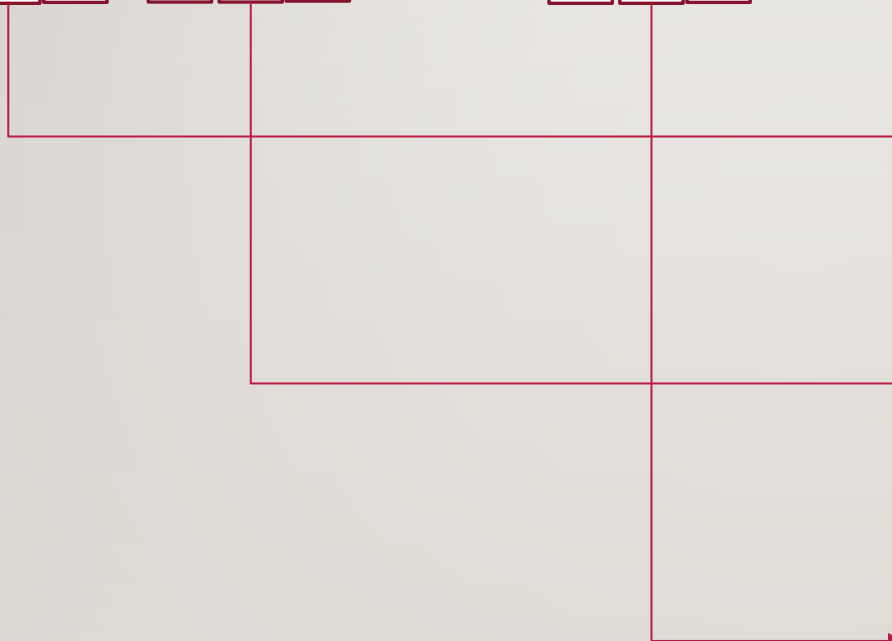
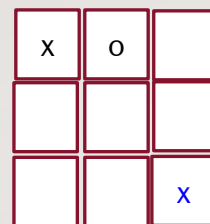
21

maximize

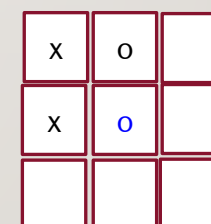
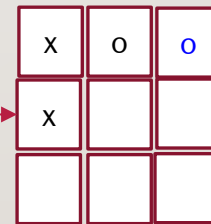
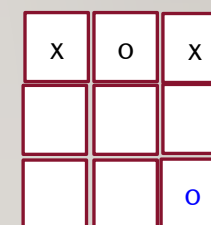
minimize



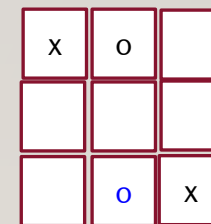
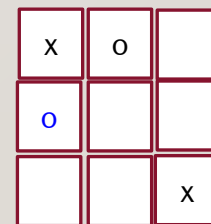
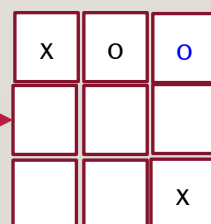
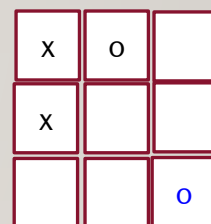
...



...



...



SOLVING TIC-TAC-TOE USING MINIMAX



อย่าลืม import

```
1 import copy
2 import random
3 import sys
```

Test

0 0

0 2

2 0

```
86 def play_tic_tac_toe():
87     board = [[' ' for _ in range(3)] for _ in range(3)]
88
89     for s in range(9): # Maximum of 9 moves
90         if s % 2 == 0:
91             row, col = map(int, input("Enter your move (row space column): ").split())
92
93             if board[row][col] != ' ':
94                 print("Cell already taken. Try again.")
95                 s -= 1 # Retry the same turn
96             else:
97                 board[row][col] = 'X'
98         else:
99             if s == 0:
100                 row, col = random.randint(0, 2), random.randint(0, 2)
101                 board[row][col] = 'O'
102             else:
103                 # AI 'O' move
104                 print("AI 'O' is thinking...")
105                 row, col = find_best_move(board)
106                 board[row][col] = 'O'
107
108         print_board(board) print(board)
109
110         if end_game(board):
111             break
```

```
113 if __name__ == "__main__":
114     play_tic_tac_toe()
```

```

57 ~ def find_best_move(board):
58     best_val = float('-inf')
59     best_move = (-1, -1)
60
61 ~     for i, j in get_empty_cells(board):
62         board_copy = copy.deepcopy(board)
63         board_copy[i][j] = 'O'
64         # find maximize AI turn first
65 ~         if is_winner(board_copy, 'O'):
66             return (i, j)
67         move_val = minimax(board_copy, 0, False)
68
69 ~         if move_val > best_val:
70             best_move = (i, j)
71             best_val = move_val
72
73     return best_move

```

```

26 def get_empty_cells(board):
27     # Get the coordinates of empty cells
28     return [(i, j) for i in range(3)
29             for j in range(3) if board[i][j] == ' ']
30

```

```

30 ~ def minimax(board, depth, maximizing_player):
31 ~     if is_winner(board, 'X'):
32         return -1
33 ~     elif is_winner(board, 'O'):
34         return 1
35 ~     elif is_full(board):
36         return 0
37
38 ~     if maximizing_player:
39         max_eval = float('-inf')
40 ~         for i, j in get_empty_cells(board):
41             board_copy = copy.deepcopy(board)
42             board_copy[i][j] = 'O'
43
44             eval = minimax(board_copy, depth + 1, False)
45             max_eval = max(max_eval, eval)
46         return max_eval
47 ~     else:
48         min_eval = float('inf')
49 ~         for i, j in get_empty_cells(board):
50             board_copy = copy.deepcopy(board)
51             board_copy[i][j] = 'X'
52             eval = minimax(board_copy, depth + 1, True)
53
54             min_eval = min(min_eval, eval)
55         return min_eval

```

25

```
11 def is_winner(board, player):
12     # Check rows, columns, and diagonals for a win
13     for i in range(3):
14         if all(board[i][j] == player for j in range(3)) or \
15             all(board[j][i] == player for j in range(3)):
16             return True
17     if all(board[i][i] == player for i in range(3)) or \
18         all(board[i][2 - i] == player for i in range(3)):
19         return True
20     return False
```

```
22 def is_full(board):
23     # Check if the board is full
24     return all(board[i][j] != ' ' for i in range(3) for j in range(3))
```

```
5 def print_board(board):
6     for row in board:
7         print(" | ".join(row))
8         print("-----")
9     print()
```

```
75 def end_game(board):
76     if is_winner(board, 'X'):
77         print("Congratulations! You won!")
78         return True
79     elif is_winner(board, 'O'):
80         print("AI 'O' wins! Better luck next time.")
81         return True
82     elif is_full(board):
83         print("It's a draw!")
84         return True
```

โค้ดยังมีปัญหา กรณีกรอกข้อมูลซ้ำ เช่น 1 1 และครั้งต่อไป เดินซ้ำ 1 1 อีกครั้ง AI จะเดินข้ามโอกาสการเดินของผู้เล่นไป

26

```
86 def play_tic_tac_toe():
87     board = [[' ' for _ in range(3)] for _ in range(3)]
88
89     for s in range(9): # Maximum of 9 moves
90         if s % 2 == 0:
91             row, col = map(int, input("Enter your move (row space column): ").split())
92
93             if board[row][col] != ' ':
94                 print("Cell already taken. Try again.")
95                 s -= 1 # Retry the same turn
96             else:
97                 board[row][col] = 'X'
98         else:
99             if s == 0:
100                 row, col = random.randint(0, 2), random.randint(0, 2)
101                 board[row][col] = 'O'
102             else:
103                 # AI 'O' move
104                 print("AI 'O' is thinking...")
105                 row, col = find_best_move(board)
106                 board[row][col] = 'O'
107
108     print_board(board)
109
110     if end_game(board):
111         break
```

ค่า s ที่เปลี่ยนแปลง ไม่สามารถ
เปลี่ยนแปลงค่า range(9) ในแต่ละ step ได้
กล่าวคือ s ลดไป 1 แต่ตอนกลับไปหาค่าสั่ง
for ค่า s ยังต่อเนื่องจาก range(9)

ช่วยหาวิธีแก้หน่อย (without ChatGPT)

27 เกมสียิบเหรียญ

- สมมติมีเหรียญ 5 เหรียญ
- ผู้เล่น 2 คน
- ผู้เล่นแต่ละคน ยิบเหรียญได้ครั้งละ 1 – 2 เหรียญเท่านั้น
- ผู้ชนะคือผู้ยิบเหรียญครั้งสุดท้าย
- จงเขียน state space representation สำหรับ minimax เพื่อสร้างโอกาสชนะ

28 เกมสืหายิบเหรียญ

- State:** (coins_remaining, current_player)

- coins_remaining:**

- The number of coins left in the game.

- current_player:**

- The player whose turn it is to make a move.

- Initial State:** The game starts with 5 coins and Player 1's turn.

- **Action:** เลือกเหรียญ 1 หรือ 2 เหรียญ

- Goal States:** These are states where the game has ended,

- either because a player has picked the last coin (resulting in a win for that player).

29 เกมสืบทอดเหรียญ กรณี 5 เหรียญ

(5, Player 1)

```
|
+--(4, Player 2)
|  |
|  +--(3, Player 1)
|  |  |
|  |  +--(2, Player 2)
|  |  |  |
|  |  |  +--(0, Player 1) -> Player 2 wins
|  |  |
|  |  +--(1, Player 2) -> Player 2 wins
|  |  |
|  |  +--(0, Player 1) -> Player 2 wins
|  +--(2, Player 1)
|  |
|  +--(0, Player 2) -> Player 1 wins
```

(5, Player 1)

```
+--(3, Player 2)
|
+--(2, Player 1)
|  |
|  +--(0, Player 1) -> Player 1 wins
|
+--(1, Player 1)
|
+--(0, Player 2) -> Player 1 wins
```

30 เกมสื้หยับเหรียญ กรณื 6 เหรียญ

(6, Player 1)

|

+--(4, Player 2)

| |

| +--(3, Player 1)

| | |

| | +--(2, Player 2)

| | | |

| | | +--(0, Player 1) -> Player 2 wins

| | |

| | +--(1, Player 2) -> Player 2 wins

| | |

| | +--(0, Player 1) -> Player 2 wins

| +--(2, Player 1)

| |

| +--(0, Player 2) -> Player 1 wins

+--(5, Player 2)

|

+ Player 2 wins

31 OPEN ISSUES

- Can we solve tic-tac-toe using alpha-beta-pruning?
- Can we role-play a good Makhos opponent?

THANK YOU

